Part 4 – Rachana: Transfer learning (Part C)

What my part is

Part C asks one question: does a CNN that has already learned from millions of ImageNet photos classify EuroSAT satellite patches better than the small CNN that Tejash trained from random numbers? I own three pieces. The first is `build_transfer_model(name, freeze)` in `src/models.py`, which loads ResNet-18 or MobileNetV2 with ImageNet weights, optionally freezes the pretrained layers, and swaps the last layer for 10 classes. The second is `notebooks/03_transfer_learning.ipynb`, which runs the four settings for seeds 0, 1 and 2, checks the results, and draws the plots. The third is the Part C report draft (`report/part_c_transfer_learning_draft.md`) together with the figures and the history JSON files the notebook writes. I did not write the training loop, the data loader or the results writer: those are shared code in `src/train.py` (Nitin) and `src/dataset.py` (Srivalli), and my notebook only calls them with the team settings. The four settings are two models times two modes: frozen (only the new head learns, learning rate 0.001) and full fine-tuning (every layer learns, learning rate 0.0001). Each setting runs for three seeds, giving 12 transfer runs, and they are compared with Tejash's 3-seed scratch CNN on the same test split. In short: full fine-tuning reaches about 98% test accuracy, frozen models reach about 87%, and the scratch CNN reaches 92.7%.

Definitions

Transfer learning. Reusing a model trained on one task as the starting point for a different task, instead of starting from random weights. We reuse ImageNet models for satellite land-use classification.

Pretraining. The first training run, done by someone else on a big dataset, that produces the starting weights. Our pretraining is ImageNet classification, done by the torchvision authors. We only download and load the result.

ImageNet and ImageNet-1K. A large photo dataset with 1,000 object classes (dogs, cars, cups). ImageNet-1K is the standard version with about 1.28 million training images. Its classes are not land-use classes, but its low-level image features are general.

Fine-tuning. Continuing to train a pretrained model on the new data, so that its weights move to fit the new task. In our project this is the "full" setting, where every layer is updated.

Frozen backbone. The pretrained part of the network (everything except the last layer) with its weights fixed. Its weights are not updated, so only the new last layer learns.

Feature extractor. The part of a network that turns an image into a vector of numbers (features). In the frozen setting the backbone is a fixed feature extractor: it gives 512 numbers for ResNet-18 and 1,280 numbers for MobileNetV2 per image.

Classifier head. The final layer(s) that turn the features into class scores. Here it is one linear layer that outputs 10 scores, one per EuroSAT class. When only the head is trained on fixed features, the method is also called a linear probe.

requires_grad. A flag on each PyTorch tensor (parameter). If it is `True`, autograd computes a gradient for it during backpropagation. If it is `False`, the tensor is treated as a constant. Setting it to `False` also saves compute and memory, because no graph is built for those layers.

Gradient and backpropagation. The gradient of the loss with respect to each weight tells us how to change it to reduce the loss. Backpropagation computes these gradients layer by layer from the output back to the input.

Learning rate. The size of each weight update step. Too large and training becomes unstable or forgets pretrained knowledge; too small and training is slow.

Adam. An optimiser that adapts the step size for each weight using running averages of its gradients. We use it with default settings and no weight decay.

Catastrophic forgetting. When a network is trained on a new task, it can overwrite the weights that held its old knowledge. Full fine-tuning with a large learning rate can destroy the useful ImageNet features, which is why we use a 10x smaller learning rate for it.

Residual connection (skip connection). A shortcut that adds a block's input to its output, `y = F(x) + x`. The shortcut gives gradients a direct path back through the network, so very deep networks train well.

ResNet-18. A 2015 CNN with 18 weighted layers (17 convolutions plus one final linear layer) arranged in four stages of residual blocks. It has about 11.7 million parameters with its original 1,000-class head.

Inverted residual. The MobileNetV2 block design. It first expands the number of channels with a 1x1 convolution, filters with a depth-wise convolution, and then projects back down to a narrow output. The skip connection links the narrow ends, which is the reverse of a classic residual block (wide, narrow, wide).

Depth-wise convolution. A convolution that filters each input channel separately, with one 3x3 filter per channel, instead of mixing all channels in each filter. It is cheap in parameters and computation.

Depthwise separable convolution. A depth-wise convolution followed by a 1x1 "pointwise" convolution that mixes channels. Together they replace a normal 3x3 convolution with far fewer weights and multiplications.

MobileNetV2. A 2018 CNN designed for phones and low compute. It stacks inverted residual blocks with depth-wise separable convolutions and has about 3.5 million parameters with its original 1,000-class head.

Parameter count. The number of numbers stored in a model's weights and biases. Total parameters count every tensor; trainable parameters count only those with `requires_grad=True`. We compute both with `p.numel()`.

BatchNorm (batch normalisation). A layer that normalises each channel of its input using the mean and variance of the current mini-batch, then applies a learnable scale (gamma) and shift (beta).

Running statistics (running_mean, running_var). Buffers that BatchNorm keeps as a moving average of batch means and variances. They are used in evaluation mode instead of batch statistics. They are not `nn.Parameter` objects, so `requires_grad=False` does not stop them from updating.

model.train() and model.eval(). Mode switches. In train mode BatchNorm uses batch statistics and updates its running buffers, and Dropout is active. In eval mode BatchNorm uses the stored running statistics and Dropout is off.

ImageNet normalisation. Subtracting the ImageNet mean and dividing by the ImageNet standard deviation per colour channel, using the values (0.485, 0.456, 0.406) and (0.229, 0.224, 0.225). Pretrained weights expect inputs on this scale.

torchvision weights enum. The `ResNet18_Weights.DEFAULT` style objects in torchvision. `DEFAULT` names the recommended weight set for that model, and a weight set also knows its own preprocessing.

Dropout. A layer that randomly zeroes a fraction of activations during training, to reduce overfitting. MobileNetV2 has one at 0.2 in its head, and we keep it.

Epoch and batch size. One epoch is one full pass over the training images. The batch size is how many images go through the network at once. We use 20 epochs and batch size 64.

Best-epoch selection. We keep the weights from the epoch with the highest validation accuracy, not the last epoch. This is a form of early stopping that uses validation data only.

Train, validation and test split. The 70/15/15 split of the 27,000 images in `data/splits.csv`. Training fits the weights, validation chooses the best epoch, and the test set is scored once at the end.

Accuracy. The share of test images whose predicted class is correct.

Macro-F1. For each class, compute the F1 score (harmonic mean of precision and recall), then take the unweighted average over the 10 classes. It treats rare and common classes equally.

Seed. A number that fixes the random generator, so that initial head weights, data order and augmentation are repeatable. We use seeds 0, 1 and 2.

Mean and sample standard deviation (ddof=1). The mean is the average of the three seeds. The sample standard deviation divides the sum of squared differences by n-1 = 2 instead of n = 3, which corrects for estimating spread from a small sample. `ddof` is the "delta degrees of freedom" argument of NumPy and pandas.

Stride and pooling. Stride is how many pixels a convolution jumps each step. A stride of 2 halves the height and width. Global average pooling (`AdaptiveAvgPool2d(1)` in our scratch CNN, and built into both torchvision models) averages each feature map to one number.

Domain shift. The gap between the data a model was trained on (ImageNet ground-level photos at 224x224) and the data we use (top-down Sentinel-2 patches at 64x64).

Data efficiency. How accuracy changes when we train on a fraction of the training images. This is Nitin's study (notebook 05). I use only the rows already in `results/metrics.csv` for context.

Checkpoint (.pth file). A saved `state_dict`, the dictionary of all weights of a model. We save the best-epoch weights into `models/` (not in git).

Linear layer (nn.Linear). A fully connected layer that computes `W x + b`. `nn.Linear(512, 10)` has 512 x 10 weights plus 10 biases, which is 5,130 parameters.

Logits and cross-entropy loss. The raw scores from the last layer. `CrossEntropyLoss` applies softmax internally and compares the probabilities with the true class.

1. Code for `build_transfer_model` (src/models.py, lines 52-78)

The file also holds Tejash's scratch CNN builder above line 52; only this function is mine. Line numbers in the table are the line numbers shown in the block.

```
def build_transfer_model(name: str, freeze: bool = True):
    """Build a pretrained ImageNet model adapted for 10 EuroSAT classes.

    name:   'resnet18' or 'mobilenet_v2'
    freeze: True keeps the backbone fixed so only the new last layer trains;
            False lets all layers fine-tune.
    """
    if name == "resnet18":
        model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    elif name == "mobilenet_v2":
        model = models.mobilenet_v2(weights=models.MobileNet_V2_Weights.DEFAULT)
    else:
        raise ValueError(f"Unsupported transfer model {name!r}; expected 'resnet18' or 'mobilenet_v2'")

    if freeze:
        # Freeze all pretrained weights first.
        for param in model.parameters():
            param.requires_grad = False

    # Replace the 1000-class ImageNet head with a new 10-class linear layer.
    # A new nn.Linear has requires_grad=True by default, so it always learns.
    if name == "resnet18":
        model.fc = nn.Linear(model.fc.in_features, 10)
    elif name == "mobilenet_v2":
        model.classifier[1] = nn.Linear(model.classifier[1].in_features, 10)

    return model
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
52	Defines the function. `name` is a string, `freeze` is a yes/no flag that defaults to `True`.	Function with a default argument	`train.py` calls `models.build_transfer_model(backbone, freeze)`, and the default matches the frozen setting. Alternative: two separate functions, one per model, or a dictionary of builders. One function keeps the call site short.
53-58	Docstring: says what the two arguments mean.	Documentation	Explains `freeze` so a teammate does not have to read the code. The type hints (`str`, `bool`) are documentation only; Python does not enforce them.
59-60	For `resnet18`, builds the torchvision ResNet-18 architecture with `weights=ResNet18_Weights.DEFAULT`, which loads the ImageNet-1K weights (cached on disk by `check_setup.py --download`).	Pretrained weights, transfer learning	Starts from features learned on 1.28 million ImageNet photos instead of random numbers. Alternatives: `weights=None` (random, the scratch case) or the old `pretrained=True` flag, which torchvision has deprecated.
61-62	Same for `mobilenet_v2` with `MobileNet_V2_Weights.DEFAULT`.	Pretrained weights	Same reason. The `DEFAULT` set for MobileNetV2 is a newer training recipe than ResNet-18's. If the exact recipe matters for a report, check `weights.meta` in torchvision.
63-64	If the name is unknown, stops with a clear error message.	Input validation, fail fast	A typo in a model name fails at once instead of building the wrong network. Alternative: return `None` (that would fail later and less clearly).
66-69	If `freeze` is true, loops over every parameter of the pretrained model and sets `requires_grad = False`.	Freezing, `requires_grad`	Stops gradients for all pretrained weights, so only the layers added after this loop can learn. The loop runs before the head is replaced, so the new head is not frozen. Alternative: keep all weights trainable and pass only head parameters to the optimiser. That also works, but it still computes gradients for the backbone, which is slower.
71-72	Comment: the new `nn.Linear` layer is trainable by default.	Default `requires_grad=True`	This is why the head always learns, even in frozen mode. We rely on the default instead of setting the flag again.
73-74	For ResNet-18, replaces `model.fc` (a `Linear(512, 1000)` layer) with `nn.Linear(model.fc.in_features, 10)`, so `Linear(512, 10)`.	Classifier head replacement	ResNet-18 pools its last features to 512 numbers. Reading `in_features` keeps the input size correct. The new layer has 512 x 10 + 10 = 5,130 parameters. Alternative: add a hidden layer before the output, which adds parameters we did not need.
75-76	For MobileNetV2, replaces only `model.classifier[1]` (the `Linear(1280, 1000)` layer) with `Linear(1280, 10)`.	Classifier head replacement, indexing into `nn.Sequential`	`classifier` is `Sequential(Dropout(0.2), Linear(1280, 1000))`. Index 1 is the linear layer, so the dropout stays. The new layer has 1280 x 10 + 10 = 12,810 parameters.
78	Returns the model.	Return value	`build_model()` in `train.py` then pairs it with the learning rate: 0.001 if frozen, 0.0001 if full. The learning-rate choice is not in this function.

Why the head is replaced rather than the whole network: the original 1,000 outputs are ImageNet object classes, which do not exist in EuroSAT. Keeping the rest of the network is the whole point of transfer learning.

2. Notebook code (`notebooks/03_transfer_learning.ipynb`)

The notebook has 16 cells. Markdown cells explain the plan. The code cells below are the ones that do work. Line numbers in each table refer to the numbered lines of that cell. The notebook text stores the Unicode "±" sign as a damaged character in two places (cell 12 and cell 14); the code is shown exactly as saved.

2.1 Cell 1: imports and device

```
from pathlib import Path
import json
import os
import sys

# Help CUDA reuse memory blocks on Windows. This does not change model behaviour.
os.environ.setdefault('PYTORCH_CUDA_ALLOC_CONF', 'expandable_segments:True')

# Make imports work whether Jupyter starts in the repo root or in the notebooks folder.
project_root = Path.cwd()
if not (project_root / 'src').is_dir():
    project_root = project_root.parent
os.chdir(project_root)
if str(project_root) not in sys.path:
    sys.path.insert(0, str(project_root))

import matplotlib.pyplot as plt
import pandas as pd
import torch

from src.train import BATCH_SIZE, MAX_EPOCHS, TRANSFER, build_model, train
from src.utils import DATA_DIR, FIGURES_DIR, METRICS_CSV, MODELS_DIR, SPLITS_CSV, get_device

device = get_device()
print('Device:', device)
print('GPU:', torch.cuda.get_device_name(0) if device.type == 'cuda' else 'not available')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Imports standard library modules: paths, JSON, environment variables, system paths.	Standard library	Used to find the project folder, read JSON and set an environment variable.
6-7	Sets `PYTORCH_CUDA_ALLOC_CONF` to `expandable_segments:True` only if it is not already set.	Environment variable, CUDA memory allocator	Reduces GPU memory fragmentation (the code comment says it helps CUDA reuse memory blocks). It does not change the maths. `setdefault` keeps any value the user already set. It must run before CUDA is first used, which it does here because `torch` is imported later.
9-15	Finds the repository root (if the kernel started in `notebooks/`, go up one folder), changes into it, and adds it to `sys.path`.	Working directory, import path	Lets `from src.train import ...` work from any starting folder. Alternative: install the project as a package with `pip install -e .`.
17-19	Imports matplotlib, pandas and torch.	Third-party libraries	Plotting, tables and tensors.
21-22	Imports the shared constants and functions: `BATCH_SIZE`, `MAX_EPOCHS`, `TRANSFER`, `build_model`, `train`, and the path constants.	Reuse of shared code	Every model uses the same batch size, epochs, split and training function, so the comparison is fair.
24-26	Picks the GPU if available and prints its name.	Device selection	Training on a GPU is much faster than on a CPU for these networks. The printed output is `cuda` and the RTX 4050 Laptop GPU. The code also runs on CPU, only slower (I did not time the CPU).

2.2 Cell 2: configuration and parameter count

```
# Initial Lap 1 transfer-learning configurations (seed 0 on the full training split).
MODEL_NAMES = [
    'resnet18_frozen',
    'resnet18_full',
    'mobilenet_v2_frozen',
```

```
    'mobilenet_v2_full',
]
INITIAL_SEED = 0
FRACTION = 1.0

FIGURES_DIR.mkdir(parents=True, exist_ok=True)
MODELS_DIR.mkdir(parents=True, exist_ok=True)
assert SPLITS_CSV.is_file(), f'Missing fixed split: {SPLITS_CSV}'
assert (DATA_DIR / 'eurosat/2750').is_dir(), 'Download EuroSAT using check_setup.py --download'

# Inspect each configuration's learning rate and parameter counts from src/train.py and src/models.py.
config_rows = []
for model_name in MODEL_NAMES:
    backbone, freeze = TRANSFER[model_name]
    model, lr = build_model(model_name)
    total_params = sum(p.numel() for p in model.parameters())
    trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
    config_rows.append({
        'model': model_name,
        'backbone': backbone,
        'freeze_backbone': freeze,
        'learning_rate': lr,
        'batch_size': BATCH_SIZE,
        'max_epochs': MAX_EPOCHS,
        'trainable_params': trainable_params,
        'total_params': total_params,
    })
    del model

config_df = pd.DataFrame(config_rows)
display(config_df)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-9	Lists the four model names, sets seed 0 for this first pass and `FRACTION = 1.0` (use the whole training split).	Experiment configuration as constants	Named constants make the notebook easy to read. The names match the keys of `TRANSFER` in `train.py`.
11-12	Creates the figure and model folders if missing.	Idempotent folder creation	Safe to run many times.
13-14	Asserts that the split file and the EuroSAT folder exist.	Fail-fast assertions	If the data is missing, the notebook stops here with a clear message instead of failing inside training.
16-22	For each name, looks up `(backbone, freeze)`, builds the model with `build_model`, and counts total and trainable parameters.	Parameter counting	This is how we checked that frozen models train only 5,130 (ResNet-18) and 12,810 (MobileNetV2) numbers. `p.numel()` counts the numbers in a tensor; the `if p.requires_grad` filter keeps only trainable ones.
23-33	Stores the row in a list and deletes the model to free GPU memory.	Memory management	Four models at once would use more GPU memory than needed for a table. Alternative: keep a single `pd.DataFrame` built from a generator.
35-36	Turns the list into a DataFrame and displays it.	Tabular summary	The displayed output lists `learning_rate` 0.001 for frozen and 0.0001 for full, and `trainable_params` 5130 / 11181642 / 12810 / 2236682.

2.3 Cell 4: seed-0 runs with the three-layer artifact guard

```
import math

seed0_runs = {}
history_path = FIGURES_DIR / 'transfer_seed0_histories.json'

existing_metrics_df = (
    pd.read_csv(METRICS_CSV)
    if METRICS_CSV.is_file() and METRICS_CSV.stat().st_size > 0
    else pd.DataFrame(columns=['model', 'fraction', 'seed', 'accuracy', 'macro_f1'])
)
saved_seed0_histories = {}
if history_path.is_file():
    if history_path.stat().st_size == 0:
        raise RuntimeError(f'Seed-0 history file is empty/corrupt: {history_path}')
    with open(history_path, 'r', encoding='utf-8') as f:
        saved_seed0_histories = json.load(f)
```

```
any_seed0_trained = False

for model_name in MODEL_NAMES:
    # 1. Check metrics CSV row for (model_name, FRACTION, INITIAL_SEED)
    row_matches = existing_metrics_df[
        (existing_metrics_df['model'] == model_name)
        & (pd.to_numeric(existing_metrics_df['fraction'], errors='coerce') == FRACTION)
        & (pd.to_numeric(existing_metrics_df['seed'], errors='coerce') == INITIAL_SEED)
    ]
    if len(row_matches) > 1:
        raise RuntimeError(
            f'Duplicate rows ({len(row_matches)}) in {METRICS_CSV} for '
            f'({model_name}, fraction={FRACTION}, seed={INITIAL_SEED}).'
        )
    has_csv = len(row_matches) == 1
    csv_acc, csv_f1 = None, None
    if has_csv:
        csv_acc = float(pd.to_numeric(row_matches['accuracy'].iloc[0], errors='coerce'))
        csv_f1 = float(pd.to_numeric(row_matches['macro_f1'].iloc[0], errors='coerce'))
        if not (
            math.isfinite(csv_acc)
            and math.isfinite(csv_f1)
            and 0.0 < csv_acc <= 1.0
            and 0.0 < csv_f1 <= 1.0
        ):
            raise RuntimeError(
                f'Invalid accuracy/macro_f1 in {METRICS_CSV} for '
                f'({model_name}, fraction={FRACTION}, seed={INITIAL_SEED}): acc={csv_acc}, f1={csv_f1}'
            )

    # 2. Check model checkpoint in models/
    ckpt_path = MODELS_DIR / f'{model_name}_frac{FRACTION:g}_seed{INITIAL_SEED}.pth'
    if ckpt_path.exists() and ckpt_path.stat().st_size == 0:
        raise RuntimeError(f'Checkpoint file is empty (0 bytes): {ckpt_path}')
    has_ckpt = ckpt_path.is_file() and ckpt_path.stat().st_size > 0

    # 3. Check seed-0 training history in transfer_seed0_histories.json
    has_hist = model_name in saved_seed0_histories
    if has_hist:
        entry = saved_seed0_histories[model_name]
        required_keys = {'best_epoch', 'val_accuracy', 'accuracy', 'macro_f1', 'seconds_per_epoch', 'history'}
        if not isinstance(entry, dict) or not required_keys.issubset(entry.keys()):
            raise RuntimeError(
                f'Invalid or incomplete history entry for {model_name} in {history_path}.'
            )
        hist = entry['history']
        for hkey in ('train_loss', 'train_accuracy', 'val_loss', 'val_accuracy'):
            vals = hist.get(hkey, []) if isinstance(hist, dict) else []
            if len(vals) != MAX_EPOCHS or not all(math.isfinite(float(v)) for v in vals):
                raise RuntimeError(
                    f'Invalid {hkey} history for {model_name} in {history_path} '
                    f'(expected {MAX_EPOCHS} finite epoch values).'
                )
        if has_csv and not (
            math.isclose(float(entry['accuracy']), csv_acc, abs_tol=1e-6)
            and math.isclose(float(entry['macro_f1']), csv_f1, abs_tol=1e-6)
        ):
            raise RuntimeError(
                f'Metric mismatch between {METRICS_CSV} and {history_path} for '
                f'({model_name}, fraction={FRACTION}, seed={INITIAL_SEED}).'
            )

    if has_csv and has_ckpt and has_hist:
        entry = saved_seed0_histories[model_name]
        seed0_runs[model_name] = {
            'model': model_name,
            'fraction': FRACTION,
            'seed': INITIAL_SEED,
            'best_epoch': int(entry['best_epoch']),
            'val_accuracy': float(entry['val_accuracy']),
            'accuracy': float(entry['accuracy']),
            'macro_f1': float(entry['macro_f1']),
            'seconds_per_epoch': float(entry['seconds_per_epoch']),
            'history': entry['history'],
        }
        print(
            f'Skip {model_name} (fraction={FRACTION}, seed={INITIAL_SEED}): '
            f'verified in {METRICS_CSV.name}, {ckpt_path.name}, and {history_path.name}'
        )
```

```
        continue

    if has_csv or has_ckpt or has_hist:
        raise RuntimeError(
            f'Inconsistent artifacts for ({model_name}, fraction={FRACTION}, seed={INITIAL_SEED}): '
            f'metrics_csv={has_csv}, checkpoint={has_ckpt} ({ckpt_path.name}), '
            f'seed0_history={has_hist} ({history_path.name}). Refusing to overwrite partial results.'
        )

    print('=' * 72)
    print(f'Starting {model_name} (fraction={FRACTION}, seed={INITIAL_SEED}, epochs={MAX_EPOCHS})')
    print('=' * 72)
    result = train(model_name, fraction=FRACTION, seed=INITIAL_SEED, epochs=MAX_EPOCHS)
    seed0_runs[model_name] = result
    any_seed0_trained = True
    existing_metrics_df = pd.read_csv(METRICS_CSV)
    print(
        f"Summary -> {model_name} | best val epoch: {result['best_epoch']} "
        f"| best val acc: {result['val_accuracy']:.4f} "
        f"| test acc: {result['accuracy']:.4f} "
        f"| test macro-F1: {result['macro_f1']:.4f} "
        f"| {result['seconds_per_epoch']:.1f} s/epoch\n"
    )

if any_seed0_trained or not history_path.is_file():
    with open(history_path, 'w', encoding='utf-8') as f:
        json.dump(
            {
                name: {
                    'best_epoch': res['best_epoch'],
                    'val_accuracy': res['val_accuracy'],
                    'accuracy': res['accuracy'],
                    'macro_f1': res['macro_f1'],
                    'seconds_per_epoch': res['seconds_per_epoch'],
                    'history': res['history'],
                }
                for name, res in seed0_runs.items()
            },
            f,
            indent=2,
        )
    print('Saved seed 0 histories to', history_path)
else:
    print('All seed 0 runs already complete and verified; preserved existing', history_path)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Imports `math`, makes an empty result dictionary and sets the path of the seed-0 history file.	Imports, dictionary as result store	The dictionary will hold one result per model.
6-10	Reads `results/metrics.csv` if it exists and is not empty; otherwise makes an empty table with the right columns.	Defensive file reading	Lets the cell run on a fresh checkout as well as a repeated run.
11-16	If the history JSON exists, checks it is not empty and loads it.	Resume from saved files	Loading saved histories is what lets a rerun skip training.
18-19	Sets a flag that records whether any new training happened.	State flag	Decides at the end whether to rewrite the JSON file.
20-21	Loops over the four model names.	Loop	One run per model.
22-31	Finds rows in the metrics table for this model, fraction and seed. Raises an error if there is more than one.	Lookup with filter, duplicate check	The metrics file must have exactly one row per run. `pd.to_numeric(..., errors='coerce')` makes the comparison tolerant of text values.
32-46	If the row exists, reads the accuracy and macro-F1 and checks both are finite and between 0 and 1.	Sanity check on stored values	Catches corrupted rows before they reach the report.
48-52	Builds the checkpoint file name (for example `resnet18_frozen_frac1_seed0.pth`), raises an error if the file is empty, and records whether it exists.	Checkpoint check	`:g` formats 1.0 as `1`, so the name matches what `train()` writes.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
54-78	If the history JSON has an entry for this model, checks it has the required keys, that each of the four curves has 20 finite values, and that its accuracy and F1 match the CSV within 1e-6.	Consistency check across three artifacts	The three records (CSV, weights, history) must agree. A mismatch stops the notebook instead of silently mixing results.
80-97	If all three artifacts exist and agree, stores the saved result and prints `Skip ...`.	Skip logic (resume)	Prevents retraining and prevents overwriting finished results.
99-104	If some but not all artifacts exist, raises an error.	Refuse partial state	A half-finished run is not overwritten silently. The user must decide.
106-111	Otherwise prints a header and calls `train()` for 20 epochs.	Calling the shared training function	`train()` loads the data, trains, picks the best epoch, scores the test set once, and adds one row to the metrics file.
112-119	Stores the result, reloads the metrics file and prints a summary line.	Result handling	The summary prints best epoch, validation accuracy, test accuracy, test macro-F1 and seconds per epoch.
121-138	If anything was trained (or the file is missing), writes all seed-0 results to `transfer_seed0_histories.json` with indent 2. Otherwise prints that it kept the existing file.	Writing a JSON history file	Stores the full per-epoch curves (float64) so later plots need no retraining.
139-140	Prints the "already complete" message.	Feedback message	Confirms that nothing was changed.

2.4 Cell 6: seed-0 table and learning curves

```
summary_rows = [
    {
        'model': model_name,
        'seed': result['seed'],
        'best_epoch': result['best_epoch'],
        'best_val_accuracy': result['val_accuracy'],
        'test_accuracy': result['accuracy'],
        'test_macro_f1': result['macro_f1'],
        'seconds_per_epoch': result['seconds_per_epoch'],
    }
    for model_name, result in seed0_runs.items()
]
seed0_df = pd.DataFrame(summary_rows)
display(seed0_df)

epochs_range = range(1, MAX_EPOCHS + 1)
fig, axes = plt.subplots(2, 2, figsize=(12, 8), sharex=True)

plot_groups = [
    ('ResNet-18', ['resnet18_frozen', 'resnet18_full'], 0),
    ('MobileNetV2', ['mobilenet_v2_frozen', 'mobilenet_v2_full'], 1),
]

for arch_title, names, row_idx in plot_groups:
    loss_ax, acc_ax = axes[row_idx, 0], axes[row_idx, 1]
    for name in names:
        hist = seed0_runs[name]['history']
        label_prefix = 'Frozen (lr=1e-3)' if name.endswith('_frozen') else 'Full (lr=1e-4)'
        loss_ax.plot(epochs_range, hist['train_loss'], linestyle='--', label=f'{label_prefix} train')
        loss_ax.plot(epochs_range, hist['val_loss'], linestyle='-', label=f'{label_prefix} val')
        acc_ax.plot(epochs_range, hist['train_accuracy'], linestyle='--', label=f'{label_prefix} train')
        acc_ax.plot(epochs_range, hist['val_accuracy'], linestyle='-', label=f'{label_prefix} val')

    loss_ax.set_title(f'{arch_title} - Loss (seed 0)')
    loss_ax.set_xlabel('Epoch')
    loss_ax.set_ylabel('Cross-entropy loss')
    loss_ax.legend()

    acc_ax.set_title(f'{arch_title} - Accuracy (seed 0)')
    acc_ax.set_xlabel('Epoch')
    acc_ax.set_ylabel('Accuracy')
    acc_ax.set_ylim(0.5, 1.0)
```

```
    acc_ax.legend()

fig.tight_layout()
curves_path = FIGURES_DIR / 'transfer_learning_curves_seed0.png'
fig.savefig(curves_path, dpi=160)
plt.show()
print('Saved learning curves to', curves_path)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-14	Builds one summary row per seed-0 model (best epoch, best validation accuracy, test accuracy, test macro-F1, seconds per epoch) and displays it.	Dictionary comprehension, DataFrame	Gives the one-table summary for seed 0. The output shows best epochs 16, 20, 13 and 17.
16-17	Sets the x-axis (epochs 1 to 20) and creates a 2 by 2 grid of plots that share the x-axis.	Matplotlib subplots	Rows are architectures, columns are loss and accuracy.
19-22	Groups the two ResNet-18 models in row 0 and the two MobileNetV2 models in row 1.	Data structure for plotting	Each architecture gets one row so frozen and full can be compared side by side.
24-32	For each model, plots training curves (dashed) and validation curves (solid), for loss and for accuracy. The label says frozen (lr=1e-3) or full (lr=1e-4).	Learning curves	A gap between training and validation curves shows overfitting. Two curves that both sit low show underfitting.
34-43	Sets titles, axis labels and legends; limits the accuracy axis to 0.5 to 1.0.	Plot formatting	Limiting the accuracy axis zooms in on the differences. The loss axis is not limited.
45-49	Lays out the figure, saves it as `transfer_learning_curves_seed0.png` at 160 dpi, shows it and prints the path.	Saving figures	Makes the figure available for the report. The saved output path points to an old `D:\` location, but the file is in this repository's `results/figures/`.

2.5 Cell 8: seeds 0, 1, 2 (resume logic)

```
import math

SEEDS = [0, 1, 2]
all_seeds_history_path = FIGURES_DIR / 'transfer_all_seeds_histories.json'

saved_all_seeds_runs = {}
if all_seeds_history_path.is_file():
    if all_seeds_history_path.stat().st_size == 0:
        raise RuntimeError(f'All-seeds history file is empty/corrupt: {all_seeds_history_path}')
    with open(all_seeds_history_path, 'r', encoding='utf-8') as f:
        all_seeds_payload = json.load(f)
    if not isinstance(all_seeds_payload, dict) or 'runs' not in all_seeds_payload:
        raise RuntimeError(f"Missing 'runs' key in {all_seeds_history_path}")
    saved_all_seeds_runs = all_seeds_payload['runs']

for model_name in MODEL_NAMES:
    for seed in SEEDS:
        existing_df = (
            pd.read_csv(METRICS_CSV)
            if METRICS_CSV.is_file() and METRICS_CSV.stat().st_size > 0
            else pd.DataFrame(columns=['model', 'fraction', 'seed', 'accuracy', 'macro_f1'])
        )

        # 1. Check metrics CSV row for (model_name, FRACTION, seed)
        row_matches = existing_df[
            (existing_df['model'] == model_name)
            & (pd.to_numeric(existing_df['fraction'], errors='coerce') == FRACTION)
            & (pd.to_numeric(existing_df['seed'], errors='coerce') == seed)
        ]
        if len(row_matches) > 1:
            raise RuntimeError(
                f'Duplicate rows ({len(row_matches)}) in {METRICS_CSV} for '
                f'({model_name}, fraction={FRACTION}, seed={seed}).'
            )
        has_csv = len(row_matches) == 1
        csv_acc, csv_f1 = None, None
```

```
        if has_csv:
            csv_acc = float(pd.to_numeric(row_matches['accuracy'].iloc[0], errors='coerce'))
            csv_f1 = float(pd.to_numeric(row_matches['macro_f1'].iloc[0], errors='coerce'))
            if not (
                math.isfinite(csv_acc)
                and math.isfinite(csv_f1)
                and 0.0 < csv_acc <= 1.0
                and 0.0 < csv_f1 <= 1.0
            ):
                raise RuntimeError(
                    f'Invalid accuracy/macro_f1 in {METRICS_CSV} for '
                    f'({model_name}, fraction={FRACTION}, seed={seed}): acc={csv_acc}, f1={csv_f1}'
                )

        # 2. Check model checkpoint in models/
        ckpt_path = MODELS_DIR / f'{model_name}_frac{FRACTION:g}_seed{seed}.pth'
        if ckpt_path.exists() and ckpt_path.stat().st_size == 0:
            raise RuntimeError(f'Checkpoint file is empty (0 bytes): {ckpt_path}')
        has_ckpt = ckpt_path.is_file() and ckpt_path.stat().st_size > 0

        # 3. Check training history in transfer_all_seeds_histories.json
        seed_key = f'seed_{seed}'
        model_runs = saved_all_seeds_runs.get(model_name, {}) if isinstance(saved_all_seeds_runs, dict) else {}
        has_hist = isinstance(model_runs, dict) and seed_key in model_runs
        if has_hist:
            run_entry = model_runs[seed_key]
            required_keys = {'best_epoch', 'val_accuracy', 'accuracy', 'macro_f1', 'seconds_per_epoch', 'history'}
            if not isinstance(run_entry, dict) or not required_keys.issubset(run_entry.keys()):
                raise RuntimeError(
                    f'Invalid or incomplete history entry for ({model_name}, {seed_key}) in {all_seeds_history_path}.'
                )
            hist = run_entry['history']
            for hkey in ('train_loss', 'train_accuracy', 'val_loss', 'val_accuracy', 'epoch_seconds'):
                vals = hist.get(hkey, []) if isinstance(hist, dict) else []
                if len(vals) != MAX_EPOCHS or not all(math.isfinite(float(v)) for v in vals):
                    raise RuntimeError(
                        f'Invalid {hkey} history for ({model_name}, {seed_key}) in {all_seeds_history_path} '
                        f'(expected {MAX_EPOCHS} finite epoch values).'
                    )
            if has_csv and not (
                math.isclose(float(run_entry['accuracy']), csv_acc, abs_tol=1e-6)
                and math.isclose(float(run_entry['macro_f1']), csv_f1, abs_tol=1e-6)
            ):
                raise RuntimeError(
                    f'Metric mismatch between {METRICS_CSV} and {all_seeds_history_path} for '
                    f'({model_name}, fraction={FRACTION}, seed={seed}).'
                )

        if has_csv and has_ckpt and has_hist:
            print(
                f'Skip {model_name} (fraction={FRACTION}, seed={seed}): '
                f'verified in {METRICS_CSV.name}, {ckpt_path.name}, and {all_seeds_history_path.name}'
            )
            continue

        if has_csv or has_ckpt or has_hist:
            raise RuntimeError(
                f'Inconsistent artifacts for ({model_name}, fraction={FRACTION}, seed={seed}): '
                f'metrics_csv={has_csv}, checkpoint={has_ckpt} ({ckpt_path.name}), '
                f'history={has_hist} ({all_seeds_history_path.name}). Refusing to overwrite partial results.'
            )

        print('=' * 72)
        print(f'Starting {model_name} (fraction={FRACTION}, seed={seed}, epochs={MAX_EPOCHS})')
        print('=' * 72)
        result = train(model_name, fraction=FRACTION, seed=seed, epochs=MAX_EPOCHS)
        print(
            f"Summary -> {model_name} (seed {seed}) | best val epoch: {result['best_epoch']} "
            f"| best val acc: {result['val_accuracy']:.4f} "
            f"| test acc: {result['accuracy']:.4f} "
            f"| test macro-F1: {result['macro_f1']:.4f} "
            f"| {result['seconds_per_epoch']:.1f} s/epoch\n"
        )
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-4	Imports `math`, sets the three seeds and the path of the all-seeds history JSON.	Configuration	Seeds 0, 1 and 2 give the three repeats for the report.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
6-14	Loads the all-seeds JSON if it exists, checks it is not empty and has a `runs` key, and stores the runs.	Resume from saved file	Same idea as cell 4, for all 12 runs.
16-17	Nested loop: each model with each seed.	Nested loop	4 models x 3 seeds = 12 runs.
18-22	Reads the metrics CSV again on every iteration.	Fresh reads	A row written by the previous `train()` call is seen by the next check. The cost is small compared with training.
24-34	Finds the metrics row for the run and raises an error if there is more than one.	Duplicate check	Same rule as cell 4.
35-49	Reads and checks accuracy and macro-F1 of the row.	Sanity check	Same rule as cell 4.
51-55	Checks the checkpoint file name and that it is not empty.	Checkpoint check	Same as cell 4, using the seed in the name.
57-83	Checks the JSON entry: required keys, five curve lists of 20 finite values (including `epoch_seconds`), and metric agreement with the CSV.	Consistency check	Same consistency idea, with one extra curve (`epoch_seconds`).
85-90	If all three agree, prints `Skip ...` and goes to the next run.	Skip logic	The 11 finished runs were skipped this way in the saved output.
92-97	If only some artifacts exist, raises an error.	Refuse partial state	Protects the 12 finished runs.
99-108	Otherwise prints headers, calls `train()`, and prints a summary line.	Shared training call	The loop body ends here.

Limitation found while reading this cell: after `train()` finishes, the cell prints a summary but does **not** add the new run to `transfer_all_seeds_histories.json`. A future rerun of a freshly trained run would then hit the "Inconsistent artifacts" error, because the CSV row and the checkpoint would exist but the history would not. This is why the JSON entry for `mobilenet_v2_full` seed 2 was assembled from the printed log lines and not written by the cell itself. The fix is one `json.dump` after the `train()` call. The saved output of this cell also prints the text "already in metrics.csv", which is not what the current code prints ("verified in ..."), so the saved outputs come from an earlier version of this cell. The notebook was not re-run from a fresh kernel after the last edits, as the log's "NOT RUN" section says.

2.6 Cell 10: verification of the 12 runs

```
# Verify and display all 12 transfer-learning runs from results/figures/transfer_all_seeds_histories.json
# and cross-check every run against results/metrics.csv.
all_seeds_history_path = FIGURES_DIR / 'transfer_all_seeds_histories.json'
with open(all_seeds_history_path, 'r', encoding='utf-8') as f:
    all_seeds_data = json.load(f)

metrics_check_df = pd.read_csv(METRICS_CSV)

all_runs_rows = []
for model_name in MODEL_NAMES:
    for seed in SEEDS:
        run_entry = all_seeds_data['runs'][model_name][f'seed_{seed}']
        hist = run_entry['history']

        # Verify all 20 epochs are present and non-empty
        assert len(hist['train_loss']) == MAX_EPOCHS
        assert len(hist['train_accuracy']) == MAX_EPOCHS
        assert len(hist['val_loss']) == MAX_EPOCHS
        assert len(hist['val_accuracy']) == MAX_EPOCHS
        assert len(hist['epoch_seconds']) == MAX_EPOCHS

        # Cross-check against results/metrics.csv
        csv_match = metrics_check_df[
            (metrics_check_df['model'] == model_name)
            & (pd.to_numeric(metrics_check_df['fraction'], errors='coerce') == FRACTION)
            & (pd.to_numeric(metrics_check_df['seed'], errors='coerce') == seed)
        ]
        assert len(csv_match) == 1, f'Missing or duplicate row in metrics.csv for {model_name} seed {seed}'
        csv_acc = float(csv_match['accuracy'].iloc[0])
```

```
        csv_f1 = float(csv_match['macro_f1'].iloc[0])

        assert abs(csv_acc - run_entry['accuracy']) == 0.0
        assert abs(csv_f1 - run_entry['macro_f1']) == 0.0
        assert round(csv_acc, 4) == run_entry['logged_test_accuracy_4dp']
        assert round(csv_f1, 4) == run_entry['logged_test_macro_f1_4dp']

        is_exact = run_entry['history_precision']['is_exact_float64_history']
        all_runs_rows.append({
            'model': model_name,
            'seed': seed,
            'fraction': run_entry['fraction'],
            'best_epoch': run_entry['best_epoch'],
            'best_val_acc': f"{run_entry['val_accuracy']:.6f}" if is_exact else f"{run_entry['val_accuracy']:.4f} (4dp log)",
            'test_acc (metrics.csv)': round(csv_acc, 6),
            'test_macro_f1 (metrics.csv)': round(csv_f1, 6),
            'logged_test_acc_4dp': f"{run_entry['logged_test_accuracy_4dp']:.4f}",
            'logged_test_f1_4dp': f"{run_entry['logged_test_macro_f1_4dp']:.4f}",
            's_per_epoch': f"{run_entry['seconds_per_epoch']:.2f}" if is_exact else f"{run_entry['seconds_per_epoch']:.1f}",
            'curve_precision': 'float64 (exact)' if is_exact else 'loss:.4f, acc:.3f (log)',
            'record_source': run_entry['record_source'],
        })

all_runs_df = pd.DataFrame(all_runs_rows)
print(f'Verified all {len(all_runs_df)} transfer-learning runs (20 epochs each) against {METRICS_CSV.name}.')
display(all_runs_df)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-5	Reads the all-seeds JSON.	File reading	Source of the 12 histories.
7	Reads the metrics CSV.	File reading	Source of the official test scores.
9-20	For each of the 12 runs, asserts that every curve has 20 epochs.	Assertions on data shape	Catches a truncated log or a run that stopped early.
22-30	Finds the single matching row in the CSV and reads its accuracy and macro-F1.	Lookup, uniqueness check	Two records of the same run must be matched one to one.
32-35	Asserts the JSON and CSV values are exactly equal (difference 0.0), and that the 4-decimal rounding equals the logged value.	Exact equality checks	For seed 0 the values come from the same float64 source, so exact equality is expected. For seeds 1 and 2 only the 4-decimal log values are available in the JSON, so the check is the rounding check.
37-50	Builds a display row with a precision label: `float64 (exact)` or `loss:.4f, acc:.3f (log)`.	Provenance labelling	Makes it visible which curves are exact and which are recovered from logs.
53-55	Builds the DataFrame, prints "Verified all 12 transfer-learning runs" and displays it.	Summary output	The printed output confirms the 12 checks passed.

2.7 Cell 12: the 3-seed summary (mean, sample SD)

```
# Three-seed statistical comparison (mean ± sample standard deviation, ddof=1) from results/metrics.csv
COMPARISON_MODELS = [
    'scratch_cnn',
    'resnet18_frozen',
    'resnet18_full',
    'mobilenet_v2_frozen',
    'mobilenet_v2_full',
]

metrics_df = pd.read_csv(METRICS_CSV)
subset_df = metrics_df[
    (metrics_df['model'].isin(COMPARISON_MODELS))
    & (pd.to_numeric(metrics_df['fraction'], errors='coerce') == FRACTION)
    & (pd.to_numeric(metrics_df['seed'], errors='coerce').isin(SEEDS))
].copy()

assert len(subset_df) == 15, f'Expected 15 rows (5 models x 3 seeds), found {len(subset_df)}'

baseline_acc_mean = subset_df.loc[subset_df['model'] == 'scratch_cnn', 'accuracy'].mean() * 100.0
baseline_f1_mean = subset_df.loc[subset_df['model'] == 'scratch_cnn', 'macro_f1'].mean() * 100.0
```

```
summary_rows = []
for model_name in COMPARISON_MODELS:
    group = subset_df[subset_df['model'] == model_name].sort_values('seed')
    acc_pct = group['accuracy'] * 100.0
    f1_pct = group['macro_f1'] * 100.0

    acc_mean = acc_pct.mean()
    acc_std = acc_pct.std(ddof=1)  # Sample standard deviation (ddof=1)
    f1_mean = f1_pct.mean()
    f1_std = f1_pct.std(ddof=1)    # Sample standard deviation (ddof=1)

    delta_acc = 0.0 if model_name == 'scratch_cnn' else (acc_mean - baseline_acc_mean)
    delta_f1 = 0.0 if model_name == 'scratch_cnn' else (f1_mean - baseline_f1_mean)

    summary_rows.append({
        'model': model_name,
        'n_seeds': len(group),
        'acc_mean_pct': round(acc_mean, 4),
        'acc_sample_sd_pct': round(acc_std, 4),
        'accuracy (mean ± sample SD)': f'{acc_mean:.2f}% ± {acc_std:.2f}%',
        'f1_mean_pct': round(f1_mean, 4),
        'f1_sample_sd_pct': round(f1_std, 4),
        'macro_f1 (mean ± sample SD)': f'{f1_mean:.2f}% ± {f1_std:.2f}%',
        'delta_acc_vs_scratch_pct': f'{delta_acc:+.2f}%',
        'delta_f1_vs_scratch_pct': f'{delta_f1:+.2f}%',
    })

summary_df = pd.DataFrame(summary_rows)
display(summary_df)
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-8	Lists the five models compared: scratch CNN and the four transfer settings.	Configuration	The five models share one table.
10-15	Reads the metrics CSV and keeps rows for these models at fraction 1.0 and seeds 0, 1, 2.	Filtering a DataFrame	Excludes the data-efficiency and ablation rows.
17	Asserts there are exactly 15 rows (5 models x 3 seeds).	Assertion	Prevents a missing or duplicated run from changing the statistics.
19-20	Computes the scratch CNN means (in percent) used as the baseline.	Baseline	All differences are measured against this number.
22-34	For each model, computes the mean and the sample standard deviation (`ddof=1`) of accuracy and macro-F1 in percent, and the difference from the baseline.	Descriptive statistics, `ddof=1`	`ddof=1` divides by n-1 = 2. This matches Tejash's Part B calculation. Alternative: `ddof=0` (population SD), which is about 18% smaller for n=3 and would not match Part B. The baseline's own difference is set to 0.
36-47	Stores the results as rounded numbers and as text in the form "mean ± SD".	Formatting	Both forms are used in the report table.
49-50	Builds and displays the summary table.	DataFrame display	Output matches the numbers in the report table in section 6 below.

2.8 Cell 14: the comparison chart

```
# Generate 3-seed comparison bar chart (Mean ± Sample SD, ddof=1) from results/metrics.csv
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from src.utils import FIGURES_DIR, METRICS_CSV

df_metrics = pd.read_csv(METRICS_CSV)
models_order = [
    'scratch_cnn',
    'resnet18_frozen',
    'resnet18_full',
    'mobilenet_v2_frozen',
    'mobilenet_v2_full',
]
display_labels = [
    'Scratch CNN\n(Baseline)',
    'ResNet-18\n(Frozen)',
```

```
    'ResNet-18\n(Full FT)',
    'MobileNetV2\n(Frozen)',
    'MobileNetV2\n(Full FT)',
]
colors = ['#4C72B0', '#DD8452', '#55A868', '#C44E52', '#8172B3']

sub_df = df_metrics[(df_metrics['model'].isin(models_order)) & (df_metrics['fraction'] == 1.0)].copy()
plot_stats = (
    sub_df.groupby('model')
    .agg(
        acc_mean=('accuracy', 'mean'),
        acc_std=('accuracy', lambda s: s.std(ddof=1)),
        f1_mean=('macro_f1', 'mean'),
        f1_std=('macro_f1', lambda s: s.std(ddof=1)),
    )
    .reindex(models_order)
)

acc_means = (plot_stats['acc_mean'] * 100).values
acc_stds = (plot_stats['acc_std'] * 100).values
f1_means = (plot_stats['f1_mean'] * 100).values
f1_stds = (plot_stats['f1_std'] * 100).values

fig, axes = plt.subplots(1, 2, figsize=(14, 5.8), dpi=160)
x = np.arange(len(models_order))
bar_width = 0.58

for ax, means, stds, metric_title, ylabel in [
    (axes[0], acc_means, acc_stds, 'Test Accuracy (%) — 3-Seed Mean ± Sample SD (ddof=1)', 'Test Accuracy (%)'),
    (axes[1], f1_means, f1_stds, 'Test Macro-F1 (%) — 3-Seed Mean ± Sample SD (ddof=1)', 'Test Macro-F1 (%)'),
]:
    bars = ax.bar(
        x, means, width=bar_width, yerr=stds, capsize=6,
        color=colors, edgecolor='#222222', linewidth=0.9, alpha=0.90,
        error_kw={'elinewidth': 1.5, 'ecolor': '#111111', 'capthick': 1.5},
    )
    ax.axhline(
        means[0], color='#4C72B0', linestyle='--', linewidth=1.2, alpha=0.75,
        label=f'Scratch CNN Baseline ({means[0]:.2f}%)',
    )
    ax.set_title(metric_title, fontsize=11.5, fontweight='bold', pad=12)
    ax.set_ylabel(ylabel, fontsize=10.5)
    ax.set_xticks(x)
    ax.set_xticklabels(display_labels, fontsize=9.5)
    ax.set_ylim(82.0, 101.2)
    ax.grid(axis='y', alpha=0.3, linestyle=':')
    ax.legend(loc='lower right', fontsize=9)
    for bar, m_val, s_val in zip(bars, means, stds):
        ax.text(
            bar.get_x() + bar.get_width() / 2.0,
            m_val + s_val + 0.35,
            f'{m_val:.2f}%\n±{s_val:.2f}%',
            ha='center', va='bottom', fontsize=8.8, fontweight='bold', color='#111111',
        )

fig.suptitle(
    'Part C Transfer Learning vs. Part B Scratch CNN Baseline on EuroSAT (fraction = 1.0, seeds 0, 1, 2)',
    fontsize=13, fontweight='bold', y=0.99,
)
plt.tight_layout(rect=[0, 0, 1, 0.94])
out_fig = FIGURES_DIR / 'transfer_vs_scratch_comparison.png'
plt.savefig(out_fig, dpi=160)
plt.show()
print(f'Saved comparison chart to: {out_fig}')
```

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
1-7	Imports numpy, pandas, matplotlib and the two paths from `src.utils`. Reads the metrics CSV.	Imports, file reading	Same data as the table, so the chart and the table cannot disagree.
8-22	Sets the model order, the short labels with line breaks, and five colours.	Plot configuration	Gives the same model order as the table.
24-34	Keeps fraction-1.0 rows of the five models and groups by model to get the mean and `ddof=1` SD of accuracy and macro-F1.	Groupby aggregation	Same statistics as cell 12, computed a second way.

Lines	What it does (simple words)	Concept	Why we did it this way (and what else could be used)
36-39	Converts all values to percent.	Units	Accuracy is stored as 0 to 1 in the CSV.
41-43	Creates a figure with two panels (14 by 5.8 inches, 160 dpi) and the x positions of the bars.	Subplots, figure size	One panel for accuracy, one for macro-F1.
45-57	For each panel, draws five bars with the SD as error bars and a dashed horizontal line at the scratch-CNN mean.	Bar chart with error bars	The dashed line makes it easy to see which models are above the baseline.
58-64	Sets the title, labels, x tick labels, the y-axis range 82 to 101.2, a dotted grid and a legend.	Plot formatting	The y-axis does not start at zero, which makes differences of a few percent visible. This exaggerates the gaps, and the report should say so (the numbers printed on the bars are the honest reading). In the saved figure the legend covers the lower parts of the two MobileNetV2 frozen and full bars.
65-71	Writes each bar's mean and ±SD above the bar.	Annotation	Readers can read the values without the table.
73-77	Sets the figure title and layout.	Layout	Keeps the title and panels from overlapping.
78-81	Saves `transfer_vs_scratch_comparison.png`, shows it and prints the path.	Saving figures	Output used in the report.

3. Architecture of the two models

ResNet-18. The network starts with a 7x7 convolution with stride 2 (64 filters), batch normalisation, ReLU and a 3x3 max-pool with stride 2. Four stages follow, with 2, 2, 2 and 2 basic blocks and 64, 128, 256 and 512 channels. A basic block is two 3x3 convolutions, each followed by batch normalisation, with a skip connection that adds the block input to its output. When the stage halves the image size or doubles the channels, the shortcut uses a 1x1 convolution to match the shapes. The skip connections matter because the gradient of `y = F(x) + x` with respect to `x` always contains a direct term of 1, so the learning signal reaches early layers even in a deep network. This is why ResNet can be trained deeper without the accuracy falling. The network ends with global average pooling (512 numbers per image) and the fully connected layer.

MobileNetV2. The first layer is a 3x3 convolution with stride 2 (32 channels). It is followed by 17 inverted residual blocks, a 1x1 convolution to 1,280 channels, global average pooling and a classifier. Each inverted residual block expands the channels with a 1x1 convolution (usually by a factor of 6), filters them with a 3x3 depth-wise convolution, and projects them back to a narrow layer with a 1x1 convolution that has no activation (the "linear bottleneck"). The skip connection is used only when the stride is 1 and the input and output widths match. The network uses depthwise separable convolutions because they are much cheaper. A normal 3x3 convolution from 128 to 128 channels needs 128 x 128 x 9 = 147,456 weights. Its depthwise separable version needs 128 x 9 = 1,152 weights for the depthwise step plus 128 x 128 = 16,384 for the pointwise step, a total of 17,536, which is about 8.4 times fewer. That saving is the reason MobileNetV2 has 2.24 million parameters against ResNet-18's 11.18 million.

Parameter counts and how the head was replaced. The counts below are the numbers printed by cell 2 and stored in the notebook output. The original torchvision models (with 1,000 ImageNet outputs) have 11,689,512 parameters for ResNet-18 and 3,504,872 for MobileNetV2. Our replacement subtracts the old last layer and adds the new one:

Model	Original head	Our head	Total parameters now	Trainable (frozen)	Trainable (full)
ResNet-18	`fc`: Linear(512, 1000) = 513,000	`fc`: Linear(512, 10) = 5,130	11,689,512 - 513,000 + 5,130 = **11,181,642**	**5,130** (0.05% of total)	11,181,642
MobileNetV2	`classifier[1]`: Linear(1280, 1000) = 1,281,000	`classifier[1]`: Linear(1280, 10) = 12,810	3,504,872 - 1,281,000 + 12,810 = **2,236,682**	**12,810** (0.57% of total)	2,236,682

The arithmetic matches the printed numbers exactly, which is a useful check that the right layer was replaced. The `in_features` of the old layer (512 and 1,280) are used for the new one, so the feature sizes line up.

4. The four settings and why the learning rates differ

Setting	Backbone trainable?	Learning rate	Trainable parameters	What learns
`resnet18_frozen`	No	0.001	5,130	Only the new 10-class head
`resnet18_full`	Yes	0.0001	11,181,642	Every layer
`mobilenet_v2_frozen`	No	0.001	12,810	Only the new 10-class head
`mobilenet_v2_full`	Yes	0.0001	2,236,682	Every layer

The head starts from random numbers, so it needs larger steps to learn quickly; 0.001 (Adam's default value, and the team's setting for the new layer) is a reasonable choice. Full fine-tuning starts from good pretrained weights. A step of 0.001 applied to every layer would move those weights far from their useful values (catastrophic forgetting), so we use 0.0001, ten times smaller. These values come from the team defaults in `build_model` (`1e-3 if freeze else 1e-4`). I did not run a learning-rate search, and I did not find a record of one, so the values are defaults and not tuned results.

The caveat. Frozen and full differ in two ways at once: which layers learn, and the learning rate. So the difference between `resnet18_frozen` and `resnet18_full` is the combined effect of both changes. We cannot say how much of the 11-point gain is due to unfreezing alone. A clean test would run frozen at 0.0001 and full at 0.001 as well, or sweep the learning rate for each setting. These runs were not done, and the report should say so.

Other settings were held fixed for all four: batch size 64, Adam with `foreach=False`, 20 epochs maximum, best-epoch selection on validation accuracy, the same 70/15/15 split and the same augmentation (random 90-degree turns and flips, applied to training images only).

What frozen mode still changes. In `train.py` the whole model is set to `model.train()` every epoch. For frozen models, the BatchNorm layers in the backbone therefore still normalise with batch statistics and update their `running_mean` and `running_var` buffers. Their learnable scale and shift are frozen, but the normalisation itself is not. So the frozen backbone is not perfectly fixed, and its features change a little during training. This is a real detail and the report mentions it.

Speed. Frozen runs are faster per epoch because autograd builds no graph through the backbone. On seed 0 the seconds per epoch were 3.19 (ResNet-18 frozen), 7.88 (ResNet-18 full), 5.36 (MobileNetV2 frozen) and 14.96 (MobileNetV2 full). MobileNetV2 has fewer parameters but is slower to train on this GPU, because depth-wise convolutions use the GPU less efficiently. The timings are wall-clock on a laptop that also had a display-driver reset in one batch, so they are noisy and should not be used as a fair efficiency claim.

5. Results

5.1 Mean and sample SD over seeds 0, 1, 2 (from `results/metrics.csv`)

Values are fraction 1.0 (full training split). Mean and sample SD use `ddof=1`. The delta is the difference in mean accuracy from the scratch CNN, in percentage points (pp).

Model	Total / trainable params	Learning rate	Test accuracy, mean ± SD (%)	Test macro-F1, mean ± SD (%)	Accuracy vs scratch (pp)
`scratch_cnn`	93,962 / 93,962	0.001	92.72 ± 0.45	92.48 ± 0.51	baseline
`resnet18_frozen`	11,181,642 / 5,130	0.001	86.85 ± 0.32	86.35 ± 0.25	-5.87
`resnet18_full`	11,181,642 / 11,181,642	0.0001	97.98 ± 0.14	97.91 ± 0.14	+5.27
`mobilenet_v2_frozen`	2,236,682 / 12,810	0.001	87.70 ± 0.29	87.27 ± 0.31	-5.02

Model	Total / trainable params	Learning rate	Test accuracy, mean ± SD (%)	Test macro-F1, mean ± SD (%)	Accuracy vs scratch (pp)
`mobilenet_v2_full`	2,236,682 / 2,236,682	0.0001	98.27 ± 0.12	98.19 ± 0.13	+5.56

Full fine-tuning improves over frozen training by 11.14 pp (ResNet-18) and 10.58 pp (MobileNetV2) in accuracy. Full fine-tuning beats the scratch CNN by 5.27 pp and 5.56 pp. Frozen training is 5.87 pp and 5.02 pp below the scratch CNN. Both full models are within 0.3 pp of each other; with 3 seeds and one split, that gap is not evidence that one architecture is better.

5.2 Every individual run (test accuracy and macro-F1, in percent)

Model	Seed 0 acc / F1 (%)	Seed 1 acc / F1 (%)	Seed 2 acc / F1 (%)
`scratch_cnn`	93.14 / 92.93	92.77 / 92.58	92.25 / 91.93
`resnet18_frozen`	86.86 / 86.39	87.16 / 86.58	86.52 / 86.09
`resnet18_full`	98.10 / 98.03	98.02 / 97.94	97.83 / 97.76
`mobilenet_v2_frozen`	87.75 / 87.26	87.38 / 86.96	87.95 / 87.59
`mobilenet_v2_full`	98.27 / 98.18	98.15 / 98.08	98.40 / 98.33

The sample SDs are small (0.12 to 0.45 pp), so the ranking of the five models is stable across these seeds. The lowest run of the full-FT models (97.83%) is still above the highest frozen run (87.95%).

5.3 Data-efficiency context (rows already in `results/metrics.csv`)

The notebooks do not contain this study; it is Nitin's part (notebook 05). I only quote the rows that are already in the metrics file, to show how the transfer and scratch models behave when training data shrinks. Only `mobilenet_v2_full` was run at reduced fractions; the frozen models and ResNet-18 were run only at 1.0.

Training fraction	Scratch CNN acc, mean ± SD (n seeds)	MobileNetV2 full acc, mean ± SD (n seeds)	Gap (pp)
0.05 (945 images)	76.48 ± 1.33 (n=4, seeds [0, 1, 2, 4])	93.69 ± 0.20 (n=3)	+17.21
0.1 (1890 images)	81.15 ± 1.04 (n=4, seeds [0, 1, 2, 4])	96.15 ± 0.33 (n=3)	+14.99
0.25 (4725 images)	85.58 ± 1.96 (n=4, seeds [0, 1, 2, 4])	97.26 ± 0.09 (n=3)	+11.68
0.5 (9450 images)	89.45 ± 1.20 (n=3, seeds [0, 1, 2])	97.76 ± 0.01 (n=3)	+8.31
1 (18900 images)	92.72 ± 0.45 (n=3, seeds [0, 1, 2])	98.27 ± 0.12 (n=3)	+5.56

Two points stand out. First, with only 5% of the training images (about 945 patches), MobileNetV2 full fine-tuning reaches about 94%, while the scratch CNN reaches about 76%. The gap is large at small fractions and narrows as data grows, which is the expected shape for pretrained features. Second, the scratch CNN rows at 5%, 10% and 25% include a seed 4 that the other rows do not have, so those scratch numbers are based on four seeds, not three. The mismatch should be fixed or stated in the report.

5.4 Learning curves (seed 0)

The figure `results/figures/transfer_learning_curves_seed0.png` has two rows (ResNet-18, MobileNetV2) and two columns (loss, accuracy), with dashed training and solid validation curves for the frozen and full settings. What it shows:

- Frozen models underfit: training accuracy stalls near 85 to 86% and the training loss stays around 0.43 (ResNet-18) and 0.48 (MobileNetV2). Training and validation curves lie close together, so the limit is the fixed features, not overfitting.
- Full models learn fast: after 2 to 3 epochs the validation accuracy is above 95%, and the final validation loss is about 0.065 to 0.08. The training loss falls to about 0.03, so there is a small gap, but validation accuracy keeps improving or stays flat.

- The seed-0 best epochs were 16, 20, 13 and 17 (for the four settings in the order of the table in 2.4). The ResNet-18 full model was still at its best on the last epoch (seed 0), so 20 epochs may limit it slightly. Across all 12 runs the best epoch of the full models ranges from 11 to 20 (ResNet-18 full: 20, 12, 11; MobileNetV2 full: 17, 16, 14), and resnet18_frozen seed 2 also peaked at epoch 20. This is not a firm finding; more epochs were not tested.

The figure `results/figures/transfer_vs_scratch_comparison.png` is the bar chart from cell 14, with the 3-seed mean and ±1 sample SD for accuracy and macro-F1, and a dashed line at the scratch-CNN mean.

5.5 History files

- `results/figures/transfer_seed0_histories.json` holds the four seed-0 runs with full float64 per-epoch curves. It was written by cell 4 from the in-memory results of `train()`, so it is exact.
- `results/figures/transfer_all_seeds_histories.json` holds all 12 runs. Its own metadata says seed 0 comes from the exact file above. The seven runs recovered from the background log `task-594.log` have losses at 4 decimal places and accuracies at 3 decimal places, because that is what the log printed. Their test accuracy and macro-F1 are the exact values from `metrics.csv`. The last run (`mobilenet_v2_full`, seed 2) was recovered from the printed output of cell 8. So the JSON is accurate for plotting, but it is not an exact float64 record for seeds 1 and 2.

Note the wording in the file: the seed-1 and seed-2 histories are log-recovered, and this is stated in the report draft's limitation (v).

6. Why ImageNet features help on satellite images (and the limits)

Why they help. Early convolution layers in CNNs learn simple detectors: edges, corners, colour contrasts and repeated textures. These detectors are not specific to ground photos. A field boundary, a forest edge, a road line and a building edge all look like edges and texture to a first-layer filter. So even a frozen first stage gives useful input to the rest of the network, and fine-tuning adds the task-specific layers. Starting from pretrained weights also makes training converge faster and more stably than starting from random weights, which is visible in the seed-0 curves: the full models reach 95% validation accuracy within 2 to 3 epochs. The data-efficiency rows show the practical effect: with 5% of the training data, the pretrained model is about 17 pp more accurate than the scratch CNN.

Why frozen features are not enough here. The frozen models learn a single linear layer on top of features from ImageNet objects. Their deeper layers encode object parts (wheels, eyes, windows of buildings) at 224 by 224 scale. EuroSAT patches are 64 by 64 pixels of top-down Sentinel-2 imagery. A 64-pixel input is reduced by a factor of 32 in ResNet-18 and MobileNetV2, so the last feature map is only 2 by 2. The deep features therefore see very little spatial structure, which is one reason a linear probe on them underperforms the scratch CNN. Fine-tuning lets the deeper filters adapt to overhead texture.

Limits of the transfer argument.

1. **Scale and resolution.** We feed 64 by 64 images directly, without resizing to 224. The ImageNet models were trained at a larger scale, so the object sizes they expect are not present in our patches. Upsampling to 224 was not tested, and it would cost about 12 times more compute per image.
2. **Bands.** EuroSAT RGB uses only the three visible Sentinel-2 bands. ImageNet has no infrared or short-wave bands, and vegetation and water are often easier to separate with near-infrared. The Part C models therefore see less information than the full 13-band dataset could provide.
3. **Normalisation.** `train.py` applies the ImageNet mean and standard deviation to the satellite images. These statistics describe photos, not satellite reflectance. The network still works because the normalised values are in a reasonable range, but the mismatch is not tuned.
4. **Data split and seeds.** One fixed 70/15/15 split and three seeds show stable results (sample SD 0.12 to 0.45 pp), but they do not measure variation across different splits or across different random patches.
5. **Frozen vs full confounding.** See section 4: the learning rate and the trainable layers change together, so the size of the gain from unfreezing is not isolated.

6. **BatchNorm in frozen mode.** The running statistics of the frozen backbone keep updating on EuroSAT batches (section 4), so "frozen" does not mean fixed in every respect.
7. **Log-recovered histories.** Seeds 1 and 2 have 4-decimal losses in the history JSON. The test scores and checkpoints are exact, so the numbers in the table are not affected.

Reproducibility notes from the code. Seeds fix the head initialisation, the batch order and the augmentation. The GPU augmentation is a different code path from the CPU loader used in Part B; Nitin's log reports his own reproduction of the scratch baseline as 92.79% against Tejash's 92.72%, and attributes the difference to the augmentation draws. I did not re-run that check, so I only quote the number.

What I would fix before the final submission. (a) Write the all-seeds history JSON from cell 8 right after each `train()` call, so a rerun cannot end in an inconsistent state. (b) Re-run the notebook from a fresh kernel (Restart and Run All) so the saved outputs match the current code. (c) Fix the damaged "±" character in cells 12 and 14. (d) Either add a frozen-at-0.0001 and a full-at-0.001 run to separate the learning-rate effect, or state clearly that it was not done. (e) Make the scratch CNN rows in the data-efficiency table use the same seeds as the transfer rows, or state the difference.